Project Report

Sentiment Analysis Using :
Prompt Engineering, Embeddings, and Fine-Tuning
On the HARD Dataset (Hotel Arabic Reviews Dataset)

Réalisé par : **Insaf GHOUAOUA**

1. Introduction

Sentiment analysis is a key domain of Natural Language Processing (NLP) that aims to extract and classify opinions expressed in textual data, typically in terms of polarity: positive, negative, or neutral. This technology is widely used across various fields, such as marketing, online reputation management, and user experience, particularly in industries like hospitality, where customer reviews play a crucial role.

This report focuses on sentiment analysis applied to Arabic hotel reviews, using the HARD dataset (Hotel Arabic Reviews Dataset). This dataset contains reviews written in Arabic, a language that presents unique challenges for NLP due to its rich morphology, complex structure, and the limited availability of suitable linguistic resources.

The main objective of this work is to compare three distinct approaches to sentiment analysis:

1. **Prompt Engineering**, which leverages advanced language models to generate predictions without requiring task-specific training.
2. **Embeddings**, which use vector representations of text for supervised classification tasks.
3. **Fine-tuning**, which involves adapting a pre-trained model to a specific task using labeled data from the dataset.

Through this report, we provide a detailed description of the necessary steps, from data preprocessing to the evaluation of different methods, in order to identify the most suitable approach for this task.

2. Développement

1. Data Preprocessing

Data preprocessing is a crucial step in any Natural Language Processing (NLP) project. It aims to improve data quality by removing irrelevant elements, standardizing formats, and making the data suitable for machine learning models. Below are the detailed preprocessing steps carried out, along with explanations.

1.1. Data Loading and Labeling

```python
import pandas as pd

data = pd.read_excel('/content/balanced-reviews.xlsx')

# Ajout d'une colonne 'Sentiment' basée sur la note
def label_sentiment(rating):
    if rating >= 4:
        return 'Positive'
    elif rating == 3:
        return 'Neutral'
    else:
        return 'Negative'

data['Sentiment'] = data['rating'].apply(label_sentiment)

# Affichage des données étiquetées
print(data[['rating', 'review', 'Sentiment']].head())
```

1.2. Removal of Non-Arabic Text

```python
import re

def remove_non_arabic(text):
    arabic_pattern = re.compile(r'[^\u0600-\u06FF\s]')
    return arabic_pattern.sub('', text)

data['Cleaned_Review'] = data['review'].apply(remove_non_arabic)

# Affichage des critiques nettoyées
print(data[['review', 'Cleaned_Review']].head())
```

1.3. Removal of Punctuation and Special Characters

```python
import string
import re

def remove_punctuation(text):
    arabic_punctuation = ['،', '؛', '؟', '٪', '…', '_', '(', ')', '[', ']', '{', '}', '<', '>']
    all_punctuation = string.punctuation + ''.join(arabic_punctuation)

    # Supprimer la ponctuation (arabe + générale)
    text = text.translate(str.maketrans('', '', all_punctuation))
    return text

data['Cleaned_Review_No_Punctuation'] = data['Cleaned_Review'].apply(remove_punctuation)

# Afficher les résultats
print(data[['Cleaned_Review', 'Cleaned_Review_No_Punctuation']].head())
```

1.4. Removal of Excessive Repeated Letters

```python
import re

def remove_repeated_letters(text):
    # Remplacer 3 répétitions ou plus par 2
    text = re.sub(r'(.)\1{2,}', r'\1\1', text)
    return text

data['Normalized_No_Repeated_Letters'] = data['Cleaned_Review_No_Punctuation'].apply(remove_repeated_letters)

# Afficher les résultats
print(data[['Cleaned_Review_No_Punctuation', 'Normalized_No_Repeated_Letters']].head())
```

1.5. Standardization of Dialectical Words into Modern StanDard Arabic

```python
# Dictionnaire de transformation du dialecte koweïtien vers l'arabe standard
dialect_to_standard_dict = {
    "شلون": "كيف",
    "زين": "جيد",
    "شفيك": "ما بك",
    "ليش": "لماذا",
    "وين": "أين",
    "شنو": "ماذا",
    "خوش": "ممتاز",
    "وايد": "كثير",
    "الحين": "الآن",
    "يبي": "يريد",
    "عشان": "لأن",
    "شوف": "انظر",
    "ماكو": "لا يوجد",
    "قاعد": "يفعل حالياً",
    "ماعليه": "لا بأس",
    "إنتي": "أنتِ",
    "إنت": "أنتَ",
    "أبوي": "أبي",
    "يمه": "أمي",
    "مالت عليك": "عار عليك",
    "كلش": "أبداً",
}

def dialect_to_standard(text):
    for dialect_word, standard_word in dialect_to_standard_dict.items():
        text = text.replace(dialect_word, standard_word)
    return text

data['Standardized_Review'] = data['Normalized_No_Repeated_Letters'].apply(dialect_to_standard)

# Afficher les résultats
print(data[['Normalized_No_Repeated_Letters', 'Standardized_Review']].head())
```

1.6. Arabic Text Normalisation

```python
def normalize_arabic(text):
    text = text.replace('ٱ', 'ا')  # Remplacer 'ٱ' par 'ا'
    text = text.replace('أ', 'ا')  # Remplacer 'أ' par 'ا'
    text = text.replace('', '')   # Supprimer 'fatha'
    text = text.replace('', '')   # 'tanwin fatha'
    text = text.replace('', '')   # 'damma'
    text = text.replace('', '')   # 'tanwin damma'
    text = text.replace('', '')   # 'kasra'
    text = text.replace('', '')   # 'tanwin kasra'
    text = text.replace('', '')   # 'sukun'
    text = text.replace('', '')   # 'shadda'
    return text

data['Normalized_Review'] = data['Standardized_Review'].apply(normalize_arabic)

# Afficher les résultats
print(data[['Standardized_Review', 'Normalized_Review']].head())
```

1.7. Saving Data to an Excel File

```python
import pandas as pd
from google.colab import files

output_data = data[['no', 'Hotel name', 'rating', 'user type', 'room type', 'nights', 'Normalized_Review', 'Sentiment']]

print(output_data.head())

output_filename = '/content/hotel_reviews_preprocessed.xlsx'
output_data.to_excel(output_filename, index=False)

# Télécharger le fichier
files.download(output_filename)

print("Le fichier Excel a été sauvegardé sous le nom 'hotel_reviews_preprocessed.xlsx'.")
```

Reference :

https://colab.research.google.com/drive/1C-z1YMChoyDa3mWpCE5ytCqtFMK6aYSO?usp=sharing

2. Sentiment Analysis Methods

2.1. Prompt Engineering :

1. Prompt engineering using ChatGPT:

We started by using ChatGPT-4o mini to generate a prompt for analyzing the sentiment of hotel reviews. The prompt we used was:

“Please analyze the sentiment of hotel reviews in the provided Excel file. The file contains columns such as 'Hotel name', 'rating', 'user type', 'room type', 'nights', and 'Normalized_Review'. For each review in the 'Normalized_Review' column, classify the sentiment as positive, neutral, or negative based on the content of the review. Also, consider the 'rating' column to refine the sentiment analysis. Once you’ve analyzed the reviews, provide the sentiment classification for each review by adding a new column 'predicted_sentiment' in the Excel file that I provided.”

ChatGPT responded by explaining the structure of the file and detailing the steps required to perform sentiment analysis. It then processed the reviews and added a new column called **predicted_sentiment**, containing the sentiment classification for each review in the Excel file.

Link to chat of ChatGPT-4o mini:
https://chatgpt.com/share/675dc5bd-79f0-800d-bec9-3eba0b29e27b

2. Data Processing, Result Comparison, and Performance Evaluation :

```
import pandas as pd
from sklearn.metrics import classification_report

file_path = '/content/testgpt.xlsx'
df = pd.read_excel(file_path)

print(df.columns)


actual_sentiment = df['Sentiment']
predicted_sentiment = df['predicted_sentiment']

actual_sentiment = actual_sentiment.str.upper()
predicted_sentiment = predicted_sentiment.str.upper()

report = classification_report(actual_sentiment, predicted_sentiment, target_names=['POSITIVE', 'NEGATIVE'])

print(report)
```

After receiving the updated Excel file from ChatGPT, which includes the **predicted_sentiment** column, we compared the obtained results with the actual review sentiments (the **Sentiment** column) as well as the sentiments predicted by ChatGPT. Using Python and the pandas library, we loaded the Excel file and compared these two columns to evaluate the performance of the sentiment analysis conducted by ChatGPT.

We also used scikit-learn to generate a classification report by comparing the true sentiment values (Sentiment column) with the predicted values (predicted_sentiment column).

3. Results Interpretation:

```
Index(['no', 'Hotel name', 'rating', 'user type', 'room type', 'nights',
       'Normalized_Review', 'Sentiment', 'predicted_sentiment'],
      dtype='object')
              precision    recall  f1-score   support

    POSITIVE       1.00      1.00      1.00        18
    NEGATIVE       1.00      1.00      1.00        81

    accuracy                           1.00        99
   macro avg       1.00      1.00      1.00        99
weighted avg       1.00      1.00      1.00        99
```

The results show that precision, recall, and F1-score are all equal to 1.00 for each category (positive and negative), meaning that ChatGPT's predictions are perfectly aligned with the true sentiments of the reviews.

An accuracy of 1.00 indicates that all predictions were correct, demonstrating that ChatGPT successfully analyzed the sentiments of the reviews.

The perfect macro and weighted averages indicate that ChatGPT handled the reviews in a balanced way, without bias toward either category.

Conclusion :

In summary, the sentiment analysis performed by ChatGPT achieved perfect results, with a complete match between the actual sentiments and the predicted labels for the hotel reviews. This demonstrates that the approach used is effective for sentiment analysis, even without relying on machine learning models. The results are consistent and reliable for the data in this file.

Reference :

https://colab.research.google.com/drive/1xWcM6wzKFOuiM8GpesvYMPzRAgLDoNAS?usp=sharing

2.2. Embeddings :

1. Tokenization and delete Stopwords

1.1 Tokenization

```
import pandas as pd
from nltk.tokenize import word_tokenize
import nltk

file_path = '/content/hotel_reviews_preprocessed.xlsx'
data = pd.read_excel(file_path)

data['Tokenized_Review'] = data['Normalized_Review'].apply(word_tokenize)

# Afficher les résultats
print(data[['Normalized_Review', 'Tokenized_Review']].head())
```

1.2 Delete Stopwords

```
from nltk.corpus import stopwords
import nltk

# Importe des stopwords de NLTK
arabic_stopwords = set(stopwords.words('arabic'))

# Liste des stopwords supplémentaires
additional_stopwords = {
    'و', 'في', 'على', 'من', 'إلى', 'عن', 'هذا', 'هذه', 'ذلك', 'تلك',
    'هو', 'هي', 'أنا', 'أنت', 'أنتم', 'هم', 'هن', 'كان', 'كانت', 'لك',
    'له', 'لها', 'لنا', 'عند', 'عليه', 'عليها', 'فيها', 'بين', 'هناك',
    'هنا', 'إذا', 'لكن', 'أو', 'بل', 'ثم', 'إن', 'أن', 'ألا', 'إلا',
    'حتى', 'كل', 'أي', 'بعض', 'بعد', 'قبل', 'أمام', 'خلف', 'تحت', 'فوق',
    'مع', 'لذلك', 'ذلك', 'مازال', 'قد',
    'عندما', 'مثل', 'إليها', 'بها', 'بهم', 'منهم', 'عليهم', 'بكم', 'أيضا'
}
# Fusionner les listes
arabic_stopwords.update(additional_stopwords)

def remove_stopwords(tokens):
    return [word for word in tokens if word not in arabic_stopwords]

data['Filtered_Review'] = data['Tokenized_Review'].apply(remove_stopwords)

# Afficher les résultats
print(data[['Tokenized_Review', 'Filtered_Review']].head())
```

2. Embeddings

In this step, we explored three different approaches for generating embeddings from Arabic hotel reviews. Embeddings are numerical representations of words that capture their semantic relationships. The three methods used are AraBERT, Word2Vec (CBOW), and FastText (Skip-Gram).

2.1 Embeddings Generation using AraBERT

```python
from transformers import AutoTokenizer, AutoModel
import torch

# Charger le tokenizer et le modèle
tokenizer = AutoTokenizer.from_pretrained('aubmindlab/bert-base-arabertv02')
model = AutoModel.from_pretrained('aubmindlab/bert-base-arabertv02')

# Fonction pour générer des embeddings
def generate_embedding(text):
    # Tokenisation et encodage
    inputs = tokenizer(text, return_tensors='pt', truncation=True, padding=True, max_length=512)
    with torch.no_grad():
        outputs = model(**inputs)
    embedding = outputs.last_hidden_state[:, 0, :].squeeze().numpy()
    return embedding

data['Embedding'] = data['Filtered_Review'].apply(lambda tokens: generate_embedding(" ".join(tokens)))

# Afficher les résultats
print(data[['Filtered_Review', 'Embedding']].head())
```

2.2 Embeddings generation using Word2Vec (CBOW)

```python
from gensim.models import Word2Vec

# Entraîner un modèle Word2Vec CBOW
word2vec_cbow_model = Word2Vec(data['Tokenized_Review'], vector_size=100, window=5, sg=0, min_count=1)

# Générer des embeddings
def generate_word2vec_cbow_embedding(tokens):
    embeddings = [word2vec_cbow_model.wv[word] for word in tokens if word in word2vec_cbow_model.wv]
    return sum(embeddings) / len(embeddings) if embeddings else [0] * 100

data['Embedding'] = data['Filtered_Review'].apply(generate_word2vec_cbow_embedding)
```

2.3 Embeddings Generation using FastText (Skip-Gram)

```
from gensim.models import FastText

# Entraîner un modèle FastText Skip-Gram
fasttext_sg_model = FastText(data['Tokenized_Review'], vector_size=100, window=5, sg=1, min_count=1)

# Générer des embeddings
def generate_fasttext_sg_embedding(tokens):
    embeddings = [fasttext_sg_model.wv[word] for word in tokens if word in fasttext_sg_model.wv]
    return sum(embeddings) / len(embeddings) if embeddings else [0] * 100

data['Embedding'] = data['Filtered_Review'].apply(generate_fasttext_sg_embedding)
```

2.5 Save Embeddings

```
from google.colab import files

def format_embedding(embedding):
    return '[' + ', '.join(f'{x:.10f}' for x in embedding) + ']'

# Création 'un DataFrame
output_data = data[['no', 'Hotel name', 'rating', 'user type', 'room type', 'nights', 'Normalized_Review', 'Sentiment']]
output_data['Embedding'] = data['Embedding'].apply(format_embedding)

print(output_data.head())

# Enregistrement des données dans un fichier Excel
output_filename = '/content/hotel_reviews_embeddings.xlsx'
output_data.to_excel(output_filename, index=False)

files.download(output_filename)

print("Le fichier Excel a été sauvegardé sous le nom 'hotel_reviews_embeddings.xlsx'.")
```

3. Training a Classifier

In this step, we trained three different methods for sentiment classification: CNN (Convolutional Neural Network), LSTM (Long Short-Term Memory), and RNN (Recurrent Neural Network). Each method uses specific neural network architectures that are well-suited to the sequential nature of textual data.

2.1 Recurrent Neural Network (RNN)

```python
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

file_path = '/content/hotel_reviews_embeddings.xlsx'
data = pd.read_excel(file_path)

# Préparer les données
X = data['Embedding']
y = data['Sentiment']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Définir le mapping des labels
label_mapping = {'Negative': 0, 'Neutral': 1, 'Positive': 2}
y_train = [label_mapping[label] for label in y_train]
y_test = [label_mapping[label] for label in y_test]

# Dataset PyTorch
class SentimentDataset(Dataset):
    def __init__(self, embeddings, labels):
        self.embeddings = [eval(emb) for emb in embeddings]
        self.labels = labels

    def __len__(self):
        return len(self.embeddings)
```

```python
    def __getitem__(self, idx):
        embedding = torch.tensor(self.embeddings[idx], dtype=torch.float32)
        label = torch.tensor(self.labels[idx], dtype=torch.long)
        return embedding, label

train_dataset = SentimentDataset(X_train, y_train)
test_dataset = SentimentDataset(X_test, y_test)

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16)

# Définir le modèle RNN
class SentimentRNN(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim, n_layers, dropout=0.2):
        super(SentimentRNN, self).__init__()
        self.rnn = nn.RNN(input_dim, hidden_dim, num_layers=n_layers, batch_first=True, dropout=dropout, nonlinearity='tanh')
        self.fc = nn.Linear(hidden_dim, output_dim)
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        # RNN Forward Pass
        out, hidden = self.rnn(x)
        # Fully Connected Layer using the last hidden state
        out = self.fc(hidden[-1])
        return self.softmax(out)

# Hyperparamètres
input_dim = len(eval(X_train.iloc[0]))
hidden_dim = 128
output_dim = 3  # Nombre de classes
n_layers = 2
```

```python
dropout = 0.2

model = SentimentRNN(input_dim, hidden_dim, output_dim, n_layers, dropout)

# Définir la fonction de perte et l'optimiseur
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Entraînement du modèle
n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    total_loss = 0
    for embeddings, labels in train_loader:
        embeddings = embeddings.unsqueeze(1)  # Ajouter une dimension pour batch_first
        optimizer.zero_grad()
        outputs = model(embeddings)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch + 1}/{n_epochs}, Loss: {total_loss / len(train_loader):.4f}")
```

2.2 Convolutional Neural Network (CNN)

```
    def __getitem__(self, idx):
        embedding = torch.tensor(self.embeddings[idx], dtype=torch.float32)
        label = torch.tensor(self.labels[idx], dtype=torch.long)
        return embedding, label

train_dataset = SentimentDataset(X_train, y_train)
test_dataset = SentimentDataset(X_test, y_test)

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16)

# Définir le modèle CNN
class SentimentCNN(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(SentimentCNN, self).__init__()
        # Reduced kernel size to 1 to match input dimension
        self.conv1 = nn.Conv1d(in_channels=input_dim, out_channels=128, kernel_size=1)
        self.conv2 = nn.Conv1d(in_channels=128, out_channels=64, kernel_size=1)
        self.pool = nn.MaxPool1d(1) # Reduced pooling kernel size for consistency
        self.fc1 = nn.Linear(64, 32)  # Changed input size to 64
        self.fc2 = nn.Linear(32, output_dim)
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        x = x.transpose(1, 2)  # Permuter les dimensions pour le CNN
        x = self.pool(torch.relu(self.conv1(x)))
        x = self.pool(torch.relu(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = torch.relu(self.fc1(x))
        x = self.fc2(x)
        return self.softmax(x)
```

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

file_path = '/content/hotel_reviews_embeddings.xlsx'
data = pd.read_excel(file_path)

# Préparer les données
X = data['Embedding']
y = data['Sentiment']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Définir le mapping des labels
label_mapping = {'Negative': 0, 'Positive': 1}
y_train = [label_mapping[label] for label in y_train]
y_test = [label_mapping[label] for label in y_test]

# Dataset PyTorch
class SentimentDataset(Dataset):
    def __init__(self, embeddings, labels):
        self.embeddings = [eval(emb) for emb in embeddings]
        self.labels = labels

    def __len__(self):
        return len(self.embeddings)
```

```python
# Hyperparamètres
input_dim = len(eval(X_train.iloc[0]))  # Dimension des embeddings
output_dim = 2  # Binaire : Positif ou Négatif

model = SentimentCNN(input_dim, output_dim)

# Définir la fonction de perte et l'optimiseur
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Entraînement du modèle
n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    total_loss = 0
    for embeddings, labels in train_loader:
        embeddings = embeddings.unsqueeze(1)
        optimizer.zero_grad()
        outputs = model(embeddings)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch + 1}/{n_epochs}, Loss: {total_loss / len(train_loader):.4f}")
```

2.3 Long Short-Term Memory (LSTM)

```python
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Charger les données
file_path = '/content/hotel_reviews_embeddings.xlsx'
data = pd.read_excel(file_path)

# Préparer les données
X = data['Embedding']
y = data['Sentiment']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Définir le mapping des labels
label_mapping = {'Negative': 0, 'Positive': 1}
y_train = [label_mapping[label] for label in y_train]
y_test = [label_mapping[label] for label in y_test]

# Dataset PyTorch
class SentimentDataset(Dataset):
    def __init__(self, embeddings, labels):
        self.embeddings = [eval(emb) for emb in embeddings]
        self.labels = labels

    def __len__(self):
        return len(self.embeddings)
```

```python
    def __getitem__(self, idx):
        embedding = torch.tensor(self.embeddings[idx], dtype=torch.float32)
        label = torch.tensor(self.labels[idx], dtype=torch.long)
        return embedding, label

train_dataset = SentimentDataset(X_train, y_train)
test_dataset = SentimentDataset(X_test, y_test)

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16)

# Définir le modèle LSTM
class SentimentLSTM(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim, num_layers=1):
        super(SentimentLSTM, self).__init__()
        self.lstm = nn.LSTM(input_dim, hidden_dim, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_dim, output_dim)
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        # LSTM retourne toutes les sorties et la dernière sortie cachée
        _, (hidden, _) = self.lstm(x)
        # On utilise la dernière couche cachée pour la classification
        hidden = hidden[-1]
        x = self.fc(hidden)
        return self.softmax(x)

# Hyperparamètres
hidden_dim = 128  # Nombre de dimensions dans les états cachés de LSTM
num_layers = 1  # Nombre de couches LSTM
input_dim = len(eval(X_train.iloc[0]))
```

```python
output_dim = 2  # Binaire : Positif ou Négatif

# Instancier le modèle
model = SentimentLSTM(input_dim, hidden_dim, output_dim, num_layers)

# Définir la fonction de perte et l'optimiseur
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Entraînement du modèle
n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    total_loss = 0
    for embeddings, labels in train_loader:
        embeddings = embeddings.unsqueeze(1)
        optimizer.zero_grad()
        outputs = model(embeddings)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch + 1}/{n_epochs}, Loss: {total_loss / len(train_loader):.4f}")
```

4. Training Classifier

2.4 Evaluation

```python
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Évaluation du modèle
model.eval()
all_preds = []
all_labels = []
with torch.no_grad():
    for embeddings, labels in test_loader:
        embeddings = embeddings.unsqueeze(1)
        outputs = model(embeddings)
        preds = torch.argmax(outputs, dim=1)
        all_preds.extend(preds.numpy())
        all_labels.extend(labels.numpy())

# Rapport de classification
print(classification_report(all_labels, all_preds, target_names=['Negative',  'Positive']))
```

Results :

1. Results of AraBERT

a. AraBERT with RNN

	precision	recall	f1-score	support
Negative	0.92	0.89	0.90	1149
Positive	0.86	0.89	0.87	851
accuracy			0.89	2000
macro avg	0.89	0.89	0.89	2000
weighted avg	0.89	0.89	0.89	2000

b. AraBERT with CNN

	precision	recall	f1-score	support
Negative	0.96	0.84	0.90	1149
Positive	0.82	0.95	0.88	851
accuracy			0.89	2000
macro avg	0.89	0.89	0.89	2000
weighted avg	0.90	0.89	0.89	2000

c. AraBERT with LSTM

	precision	recall	f1-score	support
Negative	0.89	0.93	0.91	1149
Positive	0.90	0.85	0.88	851
accuracy			0.90	2000
macro avg	0.90	0.89	0.90	2000
weighted avg	0.90	0.90	0.90	2000

2. **Results of Word2Vec**

   a. **Word2Vec with RNN**

	precision	recall	f1-score	support
Negative	0.81	0.87	0.84	1149
Positive	0.80	0.73	0.76	851
accuracy			0.81	2000
macro avg	0.81	0.80	0.80	2000
weighted avg	0.81	0.81	0.81	2000

   b. **Word2Vec with CNN**

	precision	recall	f1-score	support
Negative	0.81	0.89	0.85	1149
Positive	0.82	0.72	0.77	851
accuracy			0.81	2000
macro avg	0.82	0.80	0.81	2000
weighted avg	0.82	0.81	0.81	2000

   c. **Word2Vec with LSTM**

	precision	recall	f1-score	support
Negative	0.78	0.91	0.84	1149
Positive	0.85	0.66	0.74	851
accuracy			0.80	2000
macro avg	0.82	0.79	0.79	2000
weighted avg	0.81	0.80	0.80	2000

3. **Results of FasText**

   a. **FasText with RNN**

	precision	recall	f1-score	support
Negative	0.85	0.94	0.89	1149
Positive	0.91	0.78	0.84	851
accuracy			0.87	2000
macro avg	0.88	0.86	0.87	2000
weighted avg	0.88	0.87	0.87	2000

   b. **FasText with CNN**

	precision	recall	f1-score	support
Negative	0.86	0.94	0.90	1149
Positive	0.90	0.79	0.84	851
accuracy			0.88	2000
macro avg	0.88	0.86	0.87	2000
weighted avg	0.88	0.88	0.87	2000

c. FasText with LSTM

	precision	recall	f1-score	support
Negative	0.89	0.90	0.90	1149
Positive	0.86	0.85	0.86	851
accuracy			0.88	2000
macro avg	0.88	0.88	0.88	2000
weighted avg	0.88	0.88	0.88	2000

General Comparison :

- **Best Embedding:**

AraBERT provides the best overall performance, achieving higher scores across all evaluation metrics.

- **Best Classifier:**

LSTM is the most effective classifier in most combinations, thanks to its ability to capture long-term contextual dependencies in text.

- **Recommendation:**

The combination of AraBERT with LSTM delivers the best overall accuracy (90%) and the most balanced performance between classes.

Model Test :

In this part, we tested all the combinations of embeddings and classifiers that we had previously implemented, using the example of AraBERT with RNN.

```python
from transformers import AutoTokenizer, AutoModel
import torch

model_name = "aubmindlab/bert-base-arabertv2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
embedding_model = AutoModel.from_pretrained(model_name)

def get_embedding(text):
    tokens = tokenizer(text, return_tensors="pt", padding=True, truncation=True, max_length=128)
    with torch.no_grad():
        outputs = embedding_model(**tokens)
    return outputs.last_hidden_state[:, 0, :].squeeze(0).numpy()

def predict_sentiment(model, text):
    embedding = get_embedding(text)
    embedding_tensor = torch.tensor(embedding, dtype=torch.float32).unsqueeze(0).unsqueeze(0)
    model.eval()
    with torch.no_grad():
        output = model(embedding_tensor)
        prediction = torch.argmax(output, dim=1).item()
    label_mapping = {0: "Negative", 2: "Positive"}
    return label_mapping[prediction]

while True:
    avis = input("Entrez un avis en arabe pour prédire le sentiment (ou tapez 'quit' pour quitter) : ")
    if avis.lower() == 'quit':
        print("Sortie du programme.")
        break

    classe = predict_sentiment(model, avis)
    print(f"L'avis est classé comme : {classe}")
```

Results

```
Entrez un avis en arabe pour prédire le sentiment (ou tapez 'quit' pour quitter) : استمتعت بكل لحظة في هذا الفندق. الإفطار كان لذيذًا والخدمات ممتازة
L'avis est classé comme : Positive
Entrez un avis en arabe pour prédire le sentiment (ou tapez 'quit' pour quitter) : الفندق قديم، يحتاج إلى تجديد
L'avis est classé comme : Negative
Entrez un avis en arabe pour prédire le sentiment (ou tapez 'quit' pour quitter) : فندق جيد
L'avis est classé comme : Positive
Entrez un avis en arabe pour prédire le sentiment (ou tapez 'quit' pour quitter) : سيء للغاية
L'avis est classé comme : Negative
Entrez un avis en arabe pour prédire le sentiment (ou tapez 'quit' pour quitter) : التكييف ضعيف للغاية
L'avis est classé comme : Negative
Entrez un avis en arabe pour prédire le sentiment (ou tapez 'quit' pour quitter) : موقع الفندق ممتاز، قريب من كل شيء
L'avis est classé comme : Positive
Entrez un avis en arabe pour prédire le sentiment (ou tapez 'quit' pour quitter) : الحمام ملوث، والخدمة سيئة
L'avis est classé comme : Negative
Entrez un avis en arabe pour prédire le sentiment (ou tapez 'quit' pour quitter) :
```

Conclusion

In this step, we prepared the data by tokenizing it and removing stopwords, then we generated embeddings using different methods, including AraBERT, Word2Vec, and FastText. We then trained several classification models such as RNN, CNN, and LSTM to classify the sentiment of the reviews.

After training, we evaluated the performance of each embedding–classifier combination by testing their ability to predict sentiments using the test dataset. The predicted labels were compared to the ground truth using a classification report, providing metrics such as precision, recall, and F1-score for each class (negative, positive). Finally, we performed a practical test by converting a raw text into embeddings using one of the implemented methods and then applying a trained model to classify its sentiment.

References :

Lien vers le code complet “Embeddings_Arabert_RNN” sur Google Colab:

https://colab.research.google.com/drive/1sdmtag8FYhHNDo9AlGP5t1f5uI8YeH8r?usp=sharing

Lien vers le code complet “Embeddings_Arabert_CNN” sur Google Colab:

https://colab.research.google.com/drive/1o2icbnt55k7Jz_S7fwCrn4YzTR_lDHAw?usp=sharing

Lien vers le code complet “Embeddings_Arabert_LSTM” sur Google Colab:

https://colab.research.google.com/drive/1ubQjdkenymfccYHGdTIBG_z8v9ijiNnA?usp=sharing

Lien vers le code complet “Embeddings_ Word2Vec_RNN” sur Google Colab:

https://colab.research.google.com/drive/1oLKZ4dYTgiD92gy7wTXO9uHS04GHQKHa?usp=sharing

Lien vers le code complet “Embeddings_ Word2Vec_CNN” sur Google Colab:

https://colab.research.google.com/drive/17BgBXg77_8N7l_kS6WLehQgwzq74skmv?usp=sharing

Lien vers le code complet “Embeddings_ Word2Vec_LSTM” sur Google Colab:

https://colab.research.google.com/drive/1C1H1-a5jRC9CDnjSOn9FlJ60dRe3jU_1?usp=sharing

Lien vers le code complet “Embeddings_ FasText_RNN” sur Google Colab:

https://colab.research.google.com/drive/1tExp7CFC42VxLHVZtxKwDf7vwFQ7cKXo?usp=sharing

Lien vers le code complet “Embeddings_ FasText_CNN” sur Google Colab:

https://colab.research.google.com/drive/19EDKSbC29DmyNqCOMtYcJOwJT8kDewQK?usp=sharing

Lien vers le code complet “Embeddings_ FasText_LSTM” sur Google Colab:

https://colab.research.google.com/drive/1tLpeBt0BFw-C1jath9F5QaqNgQOIRvPR?usp=sharing

2.1. Fine-tuning :

1. FineTuning with AraGPT2

1. Importing Libraries

```
import pandas as pd
from transformers import GPT2TokenizerFast, GPT2ForSequenceClassification, Trainer, TrainingArguments
from arabert.aragpt2.grover.modeling_gpt2 import GPT2ForSequenceClassification as ArGPT2ForSequenceClassification
from arabert.preprocess import ArabertPreprocessor
from torch.utils.data import Dataset
```

2. Data Loading

```
data = pd.read_excel('/content/hotel_reviews_preprocessed.xlsx')
print(data.head())
```

3. Data Preparation

```
X = data['Normalized_Review']
y = data['Sentiment']

# Mapper les sentiments (Positive -> 1, Negative -> 0)
y = y.map({'Positive': 1, 'Negative': 0})
```

4. Loading the Pre-trained AraGPT2 Model

```
MODEL_NAME = 'aubmindlab/aragpt2-base'
arabert_prep = ArabertPreprocessor(model_name=MODEL_NAME)
model = ArGPT2ForSequenceClassification.from_pretrained(MODEL_NAME, num_labels=2)
tokenizer = GPT2TokenizerFast.from_pretrained(MODEL_NAME)

# Assigner un token de padding
tokenizer.pad_token = tokenizer.eos_token

# Définir pad_token_id dans la configuration du modèle
model.config.pad_token_id = tokenizer.pad_token_id
```

5. Creating a Custom Dataset Class

```
class TextDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        item = {key: val[idx] for key, val in self.encodings.items()}
        item['labels'] = self.labels[idx]
        return item

    def __len__(self):
        return len(self.labels)
```

6. Data Preparation

```
# Tokeniser les données
encodings = tokenizer(list(X), truncation=True, padding=True, max_length=256)
train_dataset = TextDataset(encodings, list(y))
```

7. Configuring Training Arguments

```
training_args = TrainingArguments(
    output_dir="./aragpt2-finetuned",
    num_train_epochs=10,
    # Le taux d'apprentissage
    learning_rate=5e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    warmup_steps=500,
    weight_decay=0.01,
    logging_dir="./logs",
    logging_steps=10,
    # Évaluer après chaque époque
    evaluation_strategy="epoch",
)
```

8. Crating the Training Object

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=train_dataset,
    tokenizer=tokenizer
)
```

9. Start fine-tuning

```
trainer.train()
```

10. Evaluation

```
from sklearn.metrics import classification_report
import torch

test_data = pd.read_excel('/content/hotel_reviews_test.xlsx')
X_test = test_data['Normalized_Review']
y_test = test_data['Sentiment'].map({'Positive': 1, 'Negative': 0})

encodings_test = tokenizer(list(X_test), truncation=True, padding=True, max_length=256)
test_dataset = TextDataset(encodings_test, list(y_test))

# Faire des prédictions sur le jeu de test
predictions = trainer.predict(test_dataset)

predicted_labels = predictions.predictions.argmax(axis=-1)  # Indices des classes prédites

# Afficher le classification_report
print(classification_report(y_test, predicted_labels, target_names=["Negative", "Positive"]))
```

Results :

```
              precision    recall  f1-score   support

    Negative       0.95      0.94      0.94      1190
    Positive       0.91      0.93      0.92       809

    accuracy                           0.93      1999
   macro avg       0.93      0.93      0.93      1999
weighted avg       0.93      0.93      0.93      1999
```

11. Test

```
from transformers import AutoTokenizer, GPT2ForSequenceClassification

model_name = "/content/drive/MyDrive/aragpt2-finetuned"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = GPT2ForSequenceClassification.from_pretrained(model_name)
print("Le modèle et le tokenizer ont été chargés avec succès !")
```

```python
import torch

def predict_sentiment(review):
    # Prétraiter le texte
    new_review_cleaned = arabert_prep.preprocess(review)
    inputs = tokenizer(new_review_cleaned, return_tensors="pt", padding=True, truncation=True, max_length=256)

    # Faire une prédiction
    with torch.no_grad():
        model.eval()  # Modele mode évaluation
        outputs = model(**inputs)

    # Calculer la prédiction
    logits = outputs.logits
    predicted_class = torch.argmax(logits, dim=-1).item()

    # Return sentiment prédit
    sentiment = "Positive" if predicted_class == 1 else "Negative"
    return sentiment

while True:
    new_review = input("Entrez un avis (ou tapez 'quit' pour quitter) : ")

    if new_review.lower() == 'quit':
        print("Au revoir!")
        break

    sentiment = predict_sentiment(new_review)

    print(f"Sentiment prédit : {sentiment}")
```

Results

```
Entrez un avis (ou tapez 'quit' pour quitter) : استمتعت بكل لحظة في هذا الفندق. الإفطار كان لذيذًا والخدمات ممتازة. سأعود بالتأكيد
Sentiment prédit : Positive
Entrez un avis (ou tapez 'quit' pour quitter) : الفندق قديم، يحتاج إلى تجديد
Sentiment prédit : Negative
Entrez un avis (ou tapez 'quit' pour quitter) : موقع الفندق ممتاز، قريب من كل شيء
Sentiment prédit : Positive
Entrez un avis (ou tapez 'quit' pour quitter) :
```

Conclusion

We successfully fine-tuned an AraGPT2 model to perform binary sentiment classification on Arabic hotel reviews. We adapted both the dataset and the model to our specific requirements and carefully configured the training parameters to maximize performance. The evaluation phase allowed us to assess the effectiveness of the fine-tuning process.

The next steps could include hyperparameter optimization, the use of a validation set to prevent overfitting, or integrating the model into a real-world application for real-time predictions.

Reference

Lien vers le code complet “ FineTuning_AraGPT2” sur Google Colab:

https://colab.research.google.com/drive/14grADCN5izYQU0ucXJNdutjLELfnaDN7?usp=sharing

2. FineTuning with CAMelBert

12. Importing Libraries

```
import pandas as pd
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer, TrainingArguments
from peft import get_peft_model, LoraConfig, TaskType
from torch.utils.data import Dataset
```

13. Loading Data

```
data = pd.read_excel('/content/hotel_reviews_preprocessed.xlsx')
```

14. Data preparation for fine-tuning

```
X = data['Normalized_Review']
y = data['Sentiment']

# Mapper les sentiments (Positive -> 1, Negative -> 0)
y = y.map({'Positive': 1, 'Negative': 0})
```

15. Loading the CAMeL-Lab/bert-base-arabic-camelbert-da-sentiment

```
MODEL_NAME = 'CAMeL-Lab/bert-base-arabic-camelbert-da-sentiment'

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME, num_labels=3, ignore_mismatched_sizes=True)
# Préparer QLoRA
lora_config = LoraConfig(
    task_type=TaskType.SEQ_CLS,  # Classification de séquence
    inference_mode=False,
    r=16,
    lora_alpha=32,
    lora_dropout=0.1,
    target_modules=["query", "key", "value"],
)
model = get_peft_model(model, lora_config)
```

16. Creating a Custom Dataset Class

```
class TextDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        item['labels'] = torch.tensor(self.labels[idx])
        return item

    def __len__(self):
        return len(self.labels)
```

17. Preparing Input Data for Training

```
# Tokeniser les données
encodings = tokenizer(list(X), truncation=True, padding=True, max_length=256)
train_dataset = TextDataset(encodings, list(y))
```

18. Configuring Training Arguments

```
training_args = TrainingArguments(
    output_dir="./camelbert-finetuned-qlora",
    num_train_epochs=10,  # Nombre d'époques
    learning_rate=1e-4,  # Taux d'apprentissage pour QLoRA
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    warmup_steps=500,
    weight_decay=0.01,
    logging_dir="./logs",
    logging_steps=10,
    evaluation_strategy="epoch",  # Évaluation à chaque époque
    save_strategy="epoch",  # Sauvegarde à chaque époque
    fp16=True,  # Utiliser le calcul en précision mixte
    save_total_limit=2,
    push_to_hub=False,  # Ne pas envoyer les modèles sur le Hub
)
```

19. Creating the Training Arguments

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset,
    eval_dataset=dataset,
    tokenizer=tokenizer,
)
```

20. Running fine-tuning

```
trainer.train()
```

21. Evaluation

```
from sklearn.metrics import classification_report
import torch

test_data = pd.read_excel('/content/hotel_reviews_test.xlsx')
X_test = test_data['Normalized_Review']
y_test = test_data['Sentiment'].map({'Positive': 1, 'Negative': 0})

encodings_test = tokenizer(list(X_test), truncation=True, padding=True, max_length=256)
test_dataset = TextDataset(encodings_test, list(y_test))

# Faire des prédictions sur le jeu de test
predictions = trainer.predict(test_dataset)

predicted_labels = predictions.predictions.argmax(axis=-1)  # Indices des classes prédites

# Afficher le classification_report
print(classification_report(y_test, predicted_labels, target_names=["Negative", "Positive"]))
```

Results :

```
              precision    recall  f1-score   support

    Negative       0.95      0.95      0.95      1190
    Positive       0.93      0.93      0.93       809

    accuracy                           0.94      1999
   macro avg       0.94      0.94      0.94      1999
weighted avg       0.94      0.94      0.94      1999
```

22. Test

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification

# Chargement du tokenizer et le modèle depuis Google Drive
model = AutoModelForSequenceClassification.from_pretrained(drive_model_dir)
tokenizer = AutoTokenizer.from_pretrained(drive_model_dir)

print("Modèle et tokenizer chargés depuis Google Drive.")
```

```python
# Vérifier si un GPU est disponible
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
# Fonction pour prédire le sentiment d'une critique
def predict_sentiment(review):
    # Tokeniser la revue
    inputs = tokenizer(review, return_tensors="pt", padding=True, truncation=True)
    inputs = {key: val.to(device) for key, val in inputs.items()}

    # Faire la prédiction
    outputs = model(**inputs)
    logits = outputs.logits
    predicted_class = torch.argmax(logits, dim=-1).item()

    if predicted_class == 1:
        return "Positif"
    else:
        return "Négatif"

while True:
    review = input("Entrez une critique (ou 'quit' pour quitter) : ")

    if review.lower() == 'quit':
        print("Au revoir!")
        break

    sentiment = predict_sentiment(review)
    print(f"Sentiment prédit : {sentiment}")
```

Results

```
Entrez un avis (ou tapez 'quit' pour quitter) : استمتعت بكل لحظة في هذا الفندق. الإفطار كان لذيذًا والخدمات ممتازة. سأعود بالتأكيد
Sentiment prédit : Positive
Entrez un avis (ou tapez 'quit' pour quitter) : الفندق قديم، يحتاج إلى تجديد
Sentiment prédit : Negative
Entrez un avis (ou tapez 'quit' pour quitter) : موقع الفندق ممتاز، قريب من كل شيء
Sentiment prédit : Positive
Entrez un avis (ou tapez 'quit' pour quitter) :
```

Conclusion

We used the model CAMeL-BERT, specifically **CAMeL-Lab/bert-base-arabic-camelbert-da-sentiment**, which is already fine-tuned for Arabic sentiment classification. We further adapted it to our specific task using **QLoRA**, enabling a lightweight fine-tuning process. The dataset was carefully prepared, and training parameters were optimized to achieve a high-performing model. The next steps involve hyperparameter optimization and integrating the model into a real-time prediction system.

Reference

https://colab.research.google.com/drive/1nMToYuCIB6o6CN2eQ6usil5RCgQ74FqH?usp=sharing

3. Analysis of Model Results

Comparison of the Obtained Results

1. Prompt Engineering

Méthode	Modèle	Précision (Negative)	Précision (Positive)	Rappel (Negative)	Rappel (Positive)	F1-Score (Negative)	F1-Score (Positive)	Accuracy	Macro Average	Weighted Average
Prompt Engineering	ChatGPT-4o mini	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00

2. Fine-Tuning

Méthode	Modèle	Précision (Negative)	Précision (Positive)	Rappel (Negative)	Rappel (Positive)	F1-Score (Negative)	F1-Score (Positive)	Accuracy	Macro Average	Weighted Average
Fine-Tuning	AraGPT2	0.95	0.91	0.94	0.93	0.94	0.92	0.93	0.93	0.93
	CAMelBert	0.95	0.93	0.95	0.93	0.95	0.93	0.94	0.94	0.94

3. Embedding

Méthode	Modèle	Classificateur	Précision (Negative)	Précision (Positive)	Rappel (Negative)	Rappel (Positive)	F1-Score (Negative)	F1-Score (Positive)	Accuracy	Macro Average	Weighted Average
Embedding	AraBERT	RNN	0.92	0.86	0.89	0.89	0.90	0.87	0.89	0.89	0.89
		CNN	0.96	0.82	0.84	0.95	0.90	0.88	0.89	0.89	0.89
		LSTM	0.89	0.90	0.93	0.85	0.91	0.88	0.90	0.90	0.90
	Word2Vec	RNN	0.81	0.80	0.87	0.73	0.84	0.76	0.81	0.80	0.81
		CNN	0.81	0.82	0.89	0.72	0.85	0.77	0.81	0.81	0.81
		LSTM	0.78	0.85	0.91	0.66	0.84	0.74	0.80	0.79	0.80
	FastText	RNN	0.85	0.91	0.94	0.78	0.89	0.84	0.87	0.87	0.87
		CNN	0.86	0.90	0.94	0.79	0.90	0.84	0.88	0.87	0.87
		LSTM	0.89	0.86	0.90	0.85	0.90	0.86	0.88	0.88	0.88

Conclusion

Prompt Engineering

This model achieved perfect results across all evaluation metrics, with scores of 1.00 for precision, recall, F1-score, and accuracy. Although these results are impressive, they may reflect the model configuration and dataset size. Nevertheless, this approach demonstrates exceptional performance in this specific context.

Fine-Tuning

We first performed fine-tuning using AraGPT2, achieving an accuracy of 93%, which showed a significant improvement compared to baseline models. Then, we applied fine-tuning on CAMeL-BERT, which is already fine-tuned for Arabic sentiment classification. Using QLoRA for lightweight adaptation further improved performance, leading to very high precision, recall, and accuracy. Overall, fine-tuning helped balance the performance between both classes, demonstrating its effectiveness for sentiment classification tasks.

Embedding Methods

- **AraBERT**: Results vary depending on the classifier. The LSTM model achieved the best performance with an accuracy of 0.90, a balanced F1-score across classes, and better handling of class imbalance. CNN and RNN performed slightly lower with accuracies of 0.89 and 0.88.
- **Word2Vec**: This approach is slightly less effective, with accuracies around 0.80–0.81. RNN and CNN show similar performance, while LSTM struggles more with class imbalance, highlighting its limitations in capturing complex semantic representations compared to AraBERT or FastText.
- **FastText**: FastText performs better than Word2Vec, reaching up to 0.88 accuracy. CNN and RNN provide stable and balanced results, while LSTM offers the best trade-off between precision, recall, and F1-score.

Summary

In summary, fine-tuning with AraGPT2 achieved 93% accuracy, while fine-tuning with CAMeL-BERT proved to be the most effective approach, offering the best balance between performance, generalization, and class balance. However, embedding-based models such as AraBERT and FastText combined with LSTM or CNN also demonstrated strong performance, especially for tasks requiring rich semantic representations. Finally, prompt engineering delivered perfect results in this specific setup, but it may require further validation for broader generalization.

4. Conclusion

In this practical work, we explored different sentiment analysis techniques applied to Arabic hotel review data using three distinct approaches: prompt engineering, embeddings combined with various classifiers, and fine-tuning.

The results show that each approach has its own advantages and limitations:

- **Prompt Engineering**: This approach achieved perfect results with a precision, recall, F1-score, and accuracy of 1.00. These exceptional results may be attributed to the specificity of the dataset or the configuration used. However, this method could be particularly effective in similar tasks when the data is well aligned with the model's capabilities.
- **Embeddings with Classifiers**: We tested several embeddings (AraBERT, Word2Vec, FastText) combined with classifiers such as RNN, CNN, and LSTM. The best performance was obtained using AraBERT with an LSTM classifier, reaching an accuracy of 0.90. FastText also showed strong performance with accuracy up to 0.88, while Word2Vec performed slightly lower, highlighting its limitations in capturing complex semantic representations. Although overall performance was good, differences in precision and recall between Positive and Negative classes indicate the need for better class balance.
- **Fine-Tuning**: After fine-tuning AraGPT2, we obtained an accuracy of 93%, demonstrating the effectiveness of adapting pre-trained models to specific tasks. We then fine-tuned CAMeL-BERT, which further improved performance on Arabic data. This approach achieved balanced results across both classes, with consistently high precision, recall, and F1-scores, offering better generalization and an optimal trade-off between performance and robustness.

In summary, these sentiment analysis techniques allowed us to better understand the impact of method selection and parameter tuning on model performance. Embedding-based models such as AraBERT and FastText combined with advanced classifiers like LSTM and CNN provide strong alternatives. Fine-tuning proved to be the most robust approach for real-world applications, while prompt engineering can yield outstanding results in highly specific contexts.

This practical work helped us gain deeper insight into different sentiment analysis strategies and their impact on model performance, while highlighting the importance of choosing appropriate methods and tuning parameters to achieve optimal results.

References of Tools and Models Used

- Google Colab : https://colab.google/
- Le model bert-base-arabertv2 : https://huggingface.co/aubmindlab/bert-base-arabertv2
- Le model aragpt2-base : https://huggingface.co/aubmindlab/aragpt2-base
- Le model CAMeL-Lab/bert-base-arabic-camelbert-da-sentiment : https://huggingface.co/CAMeL-Lab/bert-base-arabic-camelbert-da-sentiment
- Wandb.ai : https://wandb.ai/home